

ESPECIFICACIÓN DE INTEGRACIÓN

Ingesta por AWS S3

A25

S3

v1.0.0

Propósito: el transporte de las **entregas diarias**. Security deja el archivo en un bucket S3 y S3 **avisa solo** al backoffice, que lo procesa y monta la tabla interna. Lo comparten las seis entregas de batch (A24, A3+A4, A11, A16, A10, A7), que no lo redefinen: este documento dice **cómo llega** el archivo y cada spec dice **qué trae**. **Transporte:** S3 · bucket `nex-ingesta-<ambiente>` · un prefijo por entrega · notificación `s3:ObjectCreated:*` → **SNS** → **SQS** → worker del backoffice.

Por qué el evento y no un cron

El backoffice **no pregunta si llegó algo: se entera**. Un cron que mira una carpeta a una hora fija tiene dos problemas que no se arreglan moviendo la hora.

1. **El archivo que llega tarde espera hasta la corrida siguiente.** Si la entrega sale a las 06:20 y el cron corre a las 06:00, el pipeline trabaja un día entero con el dato de ayer sin que nada lo diga.
2. **Procesar «lo que haya» no distingue «no llegó» de «no había novedades».** Las dos cosas se ven igual desde afuera: una carpeta sin cambios.

Además, el `PutObject` de S3 es **atómico** —el objeto existe completo o no existe— y el evento se emite **cuando ya es durable**. Eso elimina el archivo a medio escribir: un transporte donde el archivo se ve mientras se sube obliga a acordar un centinela (`.done`) para saber cuándo terminar de esperar, y ese centinela es una convención más que se puede olvidar de un lado. Acá el objeto ES el centinela.

El camino del archivo



Por qué SNS en el medio y no S3 → SQS directo. Un bucket admite una sola notificación por combinación de evento y prefijo, así que encadenar un segundo consumidor —un archivador, un validador, una métrica— obligaría a tocar la configuración del bucket cada vez. Con SNS el bucket publica **una** vez y quien necesite enterarse se suscribe. El costo es un salto más y vale la pena: la configuración del bucket es la parte que Security opera y que no queremos estar cambiando.

Por qué SQS y no una llamada HTTP al backoffice. Una notificación HTTP se pierde si el backoffice está caído, desplegándose o saturado — y justamente la entrega diaria llega a una hora fija, que es cuando más probable es que coincida con una ventana de mantención. La cola retiene el evento hasta que alguien lo tome, aplica reintentos con backoff y, cuando el archivo es irrecuperable, lo deja en la **DLQ** en vez de perderlo. Un archivo que no se puede procesar tiene que quedar en algún lado donde alguien lo vea.

Convenciones del bucket

QUÉ	VALOR
Bucket	<code>nex-ingesta-<ambiente></code> (<code>dev</code> · <code>qa</code> · <code>prod</code>)
Prefijo	<code><dominio>/</code> — <code>cartera/</code> · <code>listas/</code> · <code>plataforma360/</code> · <code>otorgamiento/</code> · <code>verificacion/</code> · <code>lineas/</code>
Objeto	<code><ENTREGA>_AAAAMMDD.csv</code> — <code>CARTERA_20260916.csv</code> , <code>OTORGAMIENTO_20260916.csv</code> , ...
Filtro del evento	prefijo <code><dominio>/</code> y sufijo <code>.csv</code>
Versionado	activado — es lo que permite reprocesar una entrega puntual sin pedirla de nuevo
Cifrado	SSE-KMS, llave administrada por el dueño del bucket
Acceso público	bloqueado a nivel de cuenta y de bucket

El prefijo es el dominio y el nombre lleva la fecha. No se usa una jerarquía por fecha

(`cartera/2026/09/16/`) porque el consumidor no lista el bucket: reacciona a un evento que ya le dice la llave exacta, y una jerarquía sólo agregaría una convención más que mantener sincronizada entre las dos partes.

El contrato del evento

Lo que llega a la cola es el registro de S3, con una envoltura de SNS. El worker sólo necesita cuatro campos: **bucket**, **key**, **versionId** y **size**. Todo lo demás es contexto.

```
{
  "Records": [
    {
      "eventVersion": "2.1",
      "eventSource": "aws:s3",
      "awsRegion": "us-east-1",
      "eventTime": "2026-09-16T09:04:11.238Z",
      "eventName": "ObjectCreated:Put",
      "s3": {
        "bucket": { "name": "nex-ingesta-prod" },
        "object": {
          "key": "otorgamiento/OTORGAMIENTO_20260916.csv",
          "size": 4821993,
          "eTag": "9b2cf5e0c1a34d77b0f1d2e3a4b5c6d7",
          "versionId": "nUxJ1p9VqK2sYc0tR7mB4eH6lA3wZ8gD"
        }
      }
    }
  ]
}
```

El dominio **se deriva del prefijo de la key**, no de un campo aparte: un campo que el productor tiene que llenar es un campo que puede contradecir a la ruta, y entonces hay dos verdades sobre qué entrega es ésta.

Reglas del consumidor

1. **Idempotencia por (bucket, key, versionId)**. S3 → SNS → SQS es **al-menos-una-vez**: el mismo evento puede llegar dos veces, y con **at-least-once** no es un caso raro sino el comportamiento normal. El worker registra la terna antes de procesar y descarta lo repetido. Sin esto, una entrega de full-replace que se procesa dos veces no rompe nada, pero una de upsert sí.
2. **El orden NO está garantizado**. Si el mismo día llegan dos versiones de una entrega, el evento de la segunda puede entrar antes. Se resuelve con el **nombre del objeto** —que lleva la fecha— y, a igual fecha, con **eventTime**: gana el más reciente y el anterior se descarta con una anotación. No se resuelve con el orden de llegada, que no significa nada.
3. **Un archivo que no parsea NO se reintenta indefinidamente**. Tres intentos con backoff y a la **DLQ**, con alerta. Un CSV mal formado no se arregla reintentando y, mientras se reintenta, bloquea la cola detrás de él.
4. **La carga es transaccional por entrega**. Full-replace significa que la tabla interna queda con el archivo entero o queda como estaba: un corte a media carga deja al pipeline decidiendo con media cartera, que es peor que decidir con la de ayer.
5. **Un archivo que no llegó es un estado, no un silencio**. Cada dominio declara su hora esperada; si a esa hora no entró evento, el backoffice lo registra y la pantalla muestra la antigüedad del dato. El evento dice cuándo llegó algo, nunca cuándo faltó algo.

Permisos

- **Security** recibe un rol con `s3:PutObject` sólo sobre `nex-ingesta-<ambiente>/<dominio>/*`, sin `GetObject` ni `ListBucket`: quien deposita no necesita leer, y no poder leer es lo que evita que una credencial filtrada exponga las entregas de los demás.
 - **El worker** recibe `s3:GetObject` y `s3:GetObjectVersion` sobre el bucket, y el consumo de su cola. No tiene `PutObject`: el consumidor no escribe en el buzón del que lee.
 - **El bucket** no se expone a internet. El acceso va por **VPC endpoint**, así que el objeto no sale a la red pública en ningún tramo.
-

Reproceso

Con el versionado activo, volver a procesar una entrega no obliga a pedirla de nuevo: se reemite el evento contra el `versionId` que se quiere, desde la consola de operación del backoffice. Es la operación que resuelve el caso «la entrega estaba bien pero el worker tenía un bug»: se corrige el worker y se reproduce el mismo archivo, con la garantía de que es **el mismo bytes a bytes** y no una segunda extracción del origen, que podría traer otra cosa.

Qué queda fuera de este documento

- **Los layouts.** Ni un campo, ni un separador, ni un encoding: cada entrega los declara en su propio spec.
 - **La tabla interna.** Es lo único que la aplicación lee. El evento decide *cuándo* se monta, no *qué* se lee.
 - **Los upserts intradía (A22).** Entran por API: son correcciones puntuales dentro del día, no un archivo.
 - **AECSync (A2).** Es un stream, no una entrega de batch, y no pasa por acá.
-

Anexo · Control de versiones

Mayor = cambia lo que el sistema decide o el contrato con el servidor · **menor** = entra una sección, un campo o un criterio · **parche** = redacción, una cifra o una referencia.

VERSIÓN	FECHA	QUÉ CAMBIÓ
1.0.0	16-09-2026	Primera versión: la ingesta por S3 con notificación SNS → SQS, que reemplaza al cron y al archivo centinela.